



The Overkill Hill Method: GPT Manufacturing Cookbook (vNov25)

Introduction and Overview

Building a **Custom GPT** (a specialized AI assistant) requires more than a single prompt – it demands a carefully engineered pipeline of steps that work in harmony. The **Overkill Hill Method** is a 17-step framework to design and **manufacture** such GPTs with meticulous detail. This “cookbook” provides a tactical, standalone guide to crafting a custom GPT from scratch, combining **multi-phase prompt engineering**, external knowledge integration, dynamic memory, and alignment checks into one coherent system. The approach is direct and engineering-focused, treating prompt design with the rigor of software development (version control, testing, iteration) and the creativity of problem-solving.

Modern AI agents break down complex tasks into phases – interpreting instructions, planning, consulting tools or data, iterating, and only then responding ¹ ². Following this principle, the Overkill Hill Method decomposes the GPT creation process into a **17-step pipeline**, each with a clear objective and output. Throughout these steps, we embed advanced prompting patterns and protocols: from **SCAMPER** (a creative brainstorming framework) to **Chain-of-Thought (CoT)** reasoning and **Tree-of-Thought (ToT)** branching, from **Retrieval-Augmented Generation (RAG)** for factual grounding to **Constitutional AI** techniques for alignment, and even **self-consistency checks** to reach reasoning equilibrium. The result is a custom GPT that is **robust, knowledgeable, aligned**, and maintainable.

How to Use This Cookbook: Each step is presented with a heading and explanation, often including **Objective** (what the step achieves), **Implementation** (how to carry it out, including prompt examples or guidelines), and **Notes** (tips or context). We include **reusable prompt snippets** in code blocks for key action steps (especially Steps 5, 8, 10, 12, 13, and 16A). File types like *persona profiles*, *knowledge bases*, *pulsebooks*, and *logic extensions* are clearly defined with naming conventions and usage contexts at their relevant steps. Citations to research and best practices are provided in academic style (as footnote links) to lend credibility and allow deeper reference.

By following the Overkill Hill Method, you will create a “**master prompt pipeline**” – not just a prompt, but a *protocol* for your custom GPT ³. This pipeline is documented, versioned, and designed for re-use or future iteration. It balances structure and flexibility, ensuring the GPT can perform reliably while still leveraging the full creativity of the underlying model ⁴ ⁵. Let’s dive into the steps.

(Below is a summary of the 17 steps covered, for quick reference:)

1. **Define Objectives and Use Case Scope** – Identify exactly what the custom GPT should do.
2. **Gather Domain Knowledge & Sources** – Compile or link the knowledge it needs (knowledge files).
3. **Design the GPT’s Persona Profile** – Define the assistant’s identity, tone, and behavior.
4. **Outline the Multi-Phase Prompt Pipeline** – Plan the internal phases or stages the GPT will use to solve queries.

5. **Apply SCAMPER for Creative Completeness** – Brainstorm prompt improvements and features using SCAMPER.
6. **Incorporate Reasoning Frameworks (CoT & ToT)** – Enable step-by-step reasoning and optional branching logic.
7. **Integrate Tools and Logic Extensions** – Connect any external tools/APIs and embed special logic or rules.
8. **Implement Skeleton-of-Thought Structuring** – Have the GPT outline answers before elaborating (improves organization).
9. **Integrate Retrieval-Augmented Generation (RAG)** – Connect the GPT to knowledge files or databases for factual accuracy.
10. **Build a Verification Phase (Chain-of-Verification)** – Instruct the GPT to fact-check and validate its answers.
11. **Add Iterative Refinement & Memory (Pulsebook)** – Allow the GPT to refine answers and maintain memory of context.
12. **Apply Constitutional AI for Alignment** – Embed principles or an AI “constitution” to self-correct harmful or biased outputs.
13. **Employ Self-Consistency and Equilibrium Checks** – Use multiple reasoning paths or self-consensus to improve reliability.
14. **Assemble and Test the Full Prompt Pipeline** – Compile all components into a single system prompt or workflow and dry-run it.
15. **Versioning and Documentation** – Embed version numbers, keep change logs, and document the prompt for maintainability ⁶.
16. **Deployment and Rehydration (Step 16A)** – Deploy the custom GPT and implement rehydration logic to restore state across sessions.
17. **Governance and Continuous Improvement** – Establish usage guidelines, monitor performance, and iteratively improve the GPT over time.

Each step builds on the previous ones. By the end, you will have a fully realized custom GPT, along with the know-how to update and govern it. Now let’s cover each step in detail.

Step 1: Define Objectives and Use Case Scope

Objective: Establish a clear **blueprint** for your custom GPT. In this step, you define *what* the GPT is for, *who* it serves, and *how* it will operate at a high level. This includes the domain or subject matter, the types of questions or tasks it will handle, the target users, and any success criteria (e.g. accuracy, style, response time). Essentially, Step 1 is about capturing requirements and constraints – the “mission statement” of your GPT.

Implementation: Start by writing a short specification or **design brief** for the GPT. Treat it like defining a product’s requirements. Key points to include:

- **Use Case & Domain:** Describe the primary use case. For example, “*A legal research assistant GPT that answers questions about contract law,*” or “*A creative writing GPT that generates short stories in a given style.*” Be specific about the domain knowledge needed.
- **Audience & Interaction Mode:** Who will interact with this GPT and how? Is it a general user through chat, a developer through an API, or perhaps integrated into a tool? Defining the audience helps determine the tone and complexity of answers.

- **Capabilities & Non-goals:** List what the GPT *should be able to do* (e.g. perform math, cite sources, produce code, etc.) and any tasks it *will not do* (to avoid scope creep). This is also where you acknowledge limitations – for instance, the GPT might *not* give medical or legal advice beyond providing information, if that's a non-goal.
- **Success Criteria:** Define how you'll measure a good outcome. For example, *"answers should be factually correct with supporting references 90% of the time"*, or *"stories should adhere to the user's prompt and be at least 500 words."* These criteria will guide later refinement (Step 17).

By formalizing objectives, you prevent ambiguity later. This step does not involve writing any prompt yet – it's a planning phase. However, you can use an LLM to assist: *if unsure about the scope, ask ChatGPT to brainstorm potential requirements or edge cases*. (Just remember to critically review AI-suggested requirements for feasibility.)

Notes: This foundational step is akin to writing a project specification in software engineering. It ensures every subsequent step aligns with the initial goals. A well-defined scope also helps in designing the persona (Step 3) and knowledge base (Step 2) because you know exactly what's needed. Keep this scope document handy and consider versioning it if requirements evolve. In complex projects, **changes in objectives must be tracked** (just like prompt changes) to avoid feature creep ⁶. Clarity here sets the stage for a coherent GPT design.

Step 2: Gather Domain Knowledge & Sources

Objective: Assemble the **knowledge base** for your custom GPT. Even a powerful model needs accurate, domain-specific information to perform well. In Step 2, you will gather the resources that the GPT should have access to or be provided with. This typically results in creating one or more **knowledge files** – documents or databases of facts, reference text, or curated data that the GPT can draw from. The goal is to mitigate hallucinations and ensure up-to-date, authoritative answers by backing the GPT with real knowledge ⁷.

Implementation: Depending on your use case, domain knowledge can come in various forms:

- **Static Documents:** Collect relevant texts – e.g. manuals, Wikipedia articles, domain guidelines, FAQs, or any text corpus that covers the necessary info. For example, for a legal GPT, you might include sections of legislation or legal commentary. For a medical Q&A GPT, you might use a curated medical FAQ or textbook excerpts.
- **Knowledge Bases or APIs:** If the domain has a database (like product info, scientific data) or an API (for live data such as weather, stock prices), note these as sources. At this stage, you might download data dumps or prepare API keys/configs if needed.
- **Corporate or Private Data:** For an internal GPT (e.g., a company's HR assistant), gather internal documents or knowledge graphs relevant to that role.

Once gathered, **organize this knowledge**. In the Overkill Hill system, we use **Knowledge Files** – think of these as `.md` or `.txt` files or a vector database that the GPT will query. If using raw text files, consider naming conventions like `Knowledge_Law_v1.md` or `gpt_knowledge_contracts.txt` for clarity. If using a vector database for retrieval, prepare an embedding pipeline to ingest these documents (the actual retrieval integration is handled in Step 9, but the content must be ready now).

Usage Context: These knowledge files will be used in the prompt pipeline likely via retrieval (Step 9) or direct priming. For now, ensure they are **clean and concise**. It may help to preprocess them: remove irrelevant sections, add indexing (for retrieval), or even summarize if the text is very long (though be careful – summarization might omit needed details).

Example: If building the legal research GPT, you might create a file `contract_law_refs.md` containing key statutes, case law summaries, and definitions. If the GPT will cite sources, format this file with clear section titles or IDs so that retrieval can identify a source and the GPT can reference it (e.g., “[Section 2.1 of contract law reference]”).

Notes: Incorporating an external knowledge base is crucial for **factual accuracy**. Large models have training cut-offs and may not know recent or niche info. **Retrieval-Augmented Generation (RAG)** addresses this by having the model fetch relevant text from an external source and use it to craft an answer ⁷ ⁸. By preparing knowledge files now, you are laying the groundwork for Step 9 where we integrate retrieval. Keep track of sources for each piece of knowledge – you might embed citations in the text or maintain a separate mapping, so that the GPT can provide references if needed (you’ll see more in Step 10 on verification and citations). Remember to version your knowledge files as well; they are part of your GPT’s configuration. A filename with a version (e.g. `_v1`) or a change log entry can record updates to the knowledge base over time.

Step 3: Design the GPT’s Persona Profile

Objective: Define *who* the GPT is and *how* it communicates – in other words, create a **Persona Profile** for your AI. The persona encompasses the assistant’s role, tone, style, and behavioral guidelines. A well-defined persona ensures consistency in responses and alignment with user expectations (friendly vs. formal, expert vs. beginner-friendly, etc.). It’s essentially the custom GPT’s “character bible,” encoding its identity and voice.

Implementation: Write a **persona document** that you will later include as part of the system prompt or initial context. This profile typically covers:

- **Role and Expertise:** Describe the AI as if it were a real person or specialist. For example: “You are **Lexi**, a virtual legal analyst with 10 years of experience in contract law.” This grounds the model in a role. If domain-specific, mention credentials or background (even if fictional, it sets context).
- **Tone and Style:** Specify the tone (e.g. polite, enthusiastic, factual, empathetic). Also note stylistic preferences: does it use technical jargon or layman terms? Short sentences or elaborate explanations? For instance, “Lexi speaks in a formal, professional tone, providing detailed explanations with legal terminology, but also offers plain-English summaries for clarity.”
- **Behavior and Etiquette:** Set boundaries and manners. Should the GPT always cite sources? Does it ask clarifying questions if unsure? Should it refuse certain requests (this ties into governance later, but basic refusal rules can be stated here)? Example: “Lexi will not give definitive legal advice but will provide information and suggest consulting a lawyer for critical decisions. Lexi always maintains confidentiality and never uses offensive language.”
- **Persona Motivations or Quirks (optional):** Sometimes adding a bit of “personality” can make interactions more engaging. Perhaps the GPT has a teaching style (“enjoys explaining concepts via analogies”) or a cautious nature (“prefers to double-check facts using provided references”). Be careful to keep it subtle and aligned with purpose – the persona should not override factuality or user instructions, but it can shape how the answers are presented.

Create this persona profile in a file, e.g., `persona_LexiLaw_v1.txt`. This is a static piece of text that will later be inserted into the prompt (likely as system message or at the top of your prompt script). Use clear, declarative sentences and even bullet points for different aspects to ensure the model can parse it. You might format certain *do's and don'ts* as bullet points for emphasis.

Usage Context: The persona profile is essentially part of the system-level prompting. In the final assembly (Step 14), it will precede user queries. For instance, your final prompt might start with something like: *"You are Lexi, a legal analyst AI... [tone guidelines]... [behavior guidelines]..."* so that every conversation starts with the model "in character." The profile sets a default context that persists unless overridden by later instructions.

Notes: Consistency is key – by nailing the persona early, you avoid the GPT responding in unpredictable ways. It's also a place to bake in some **alignment/safety rules** gently. For example, including "Lexi does not engage in political opinion and remains neutral" or "Lexi refuses to assist with any illegal activity" preemptively sets boundaries (though stronger safety will come in Step 12). The persona can also help avoid style drift. In a multi-turn conversation, if the model starts getting too informal or off-track, the system-level persona reminder keeps it on course.

Remember that persona is a double-edged sword: an overly verbose persona eats into the prompt budget, and too strong a persona might make the model inflexible. Striking the right balance is important – provide enough detail to guide style and ethics, but not so much that it stifles the model's ability to follow user instructions. You can **iterate on the persona** as you test the GPT: if you see undesired behavior, you might come back and add a line to the persona profile to correct it (and bump the version, e.g., v1 -> v1.1 as a patch ⁹).

Step 4: Outline the Multi-Phase Prompt Pipeline

Objective: Design the **internal workflow** that your GPT will use to answer questions or perform tasks. Instead of a single-shot response, complex tasks benefit from a multi-phase approach ¹ ¹⁰. In this step, you outline the sequence of reasoning or action phases the GPT should go through (mentally) before producing a final answer. The outcome is a **prompt pipeline blueprint**: a structured plan enumerating each phase or step the GPT will execute (e.g., understanding, planning, verifying, etc.). This provides the skeleton for the master prompt you'll assemble later.

Implementation: Given your use case (from Step 1) and the resources/persona from Steps 2-3, decide how to break down a typical query. Common phases (inspired by agent frameworks and prior art) include ¹¹:

- **Phase 1: Understand the Query and Context.** The GPT should first interpret what the user is asking, possibly rephrasing it or identifying sub-tasks. This ensures it's on the right track ¹². (E.g., *"The user asks X; to answer, I need to find Y and consider Z."*)
- **Phase 2: Plan a Solution or Approach.** Next, the GPT outlines steps or a strategy to solve the query ¹³. Think of this as the model's to-do list before final answer. (E.g., *"1. Check knowledge file for Y, 2. Verify fact Z, 3. Draft answer covering A, B, C."*)
- **Phase 3: Retrieve or Research (if needed).** If external info is needed (very likely, since we have knowledge files), have a phase where the GPT explicitly calls for that info. In practice, this could be

implemented via a tool or by the prompt instructing: “Now search your knowledge base for [topic].” We will refine this in Step 9 (RAG integration), but include a placeholder in the plan.

- **Phase 4: Verify and Fact-Check.** A dedicated step to check the accuracy of information gathered or the assumptions made ¹⁴ ¹⁵ . The GPT might pose questions to itself like “Is X true? Did I get Y’s definition right?” – similar to a checklist.
- **Phase 5: Synthesize and Draft Answer.** The GPT now attempts an answer based on the plan and verified info. This might be the first full answer draft.
- **Phase 6: Refine/Review.** The GPT reviews its draft for any inconsistencies or missed points, possibly comparing against the query requirements (Step 1 criteria). It revises if needed ¹⁶ ¹⁷ .
- **Phase 7: Finalize Answer.** The GPT presents the final answer, formatted properly, and possibly accompanied by citations or notes as required ¹⁰ ¹⁸ .

The above is a generic template; you should customize phases to your context. You may not need all, or you might need additional ones (e.g., a “**Creative Brainstorm**” phase for a story-writing GPT, or a “Safety Check” phase explicitly before final answer for a highly regulated domain). The key is sequencing: each phase’s output feeds the next. Document this pipeline clearly, as a numbered list of phase names with their purpose.

This outline effectively serves as the structure for your master prompt. In the final implementation, each phase will correspond to either a section of the prompt or an actual turn in a multi-turn execution. For example, some developers implement these phases as chain-of-thought in one prompt, separated by special tokens or comments. Others implement them as distinct prompts in a loop (especially if using an agent orchestrator). In this cookbook, we assume a single unified prompt document that the model processes sequentially (the model “internalizes” the phases) ¹⁹ .

Example (for a legal Q&A GPT):

1. *Understand:* Restate the legal question, identify jurisdiction or law area.
2. *Plan:* List steps (check relevant statute in knowledge file, find relevant cases, then form an explanation).
3. *Retrieve:* Pull text from knowledge base on that statute and any similar cases.
4. *Verify:* Ensure the statute text addresses the question; double-check that case law supports the interpretation.
5. *Draft:* Compose an answer with explanations and legal references.
6. *Refine:* Self-review the answer for missing pieces or unclear language; improve clarity.
7. *Finalize:* Provide the answer, with citations to the knowledge sources used.

Notes: This step is informed by frameworks like Multi-Component Prompting (MCP) which formalize breaking tasks into components like plan, verify, etc. ¹¹ . By outlining now, you make the next steps (where we fill in details for each phase) much easier. It’s akin to writing pseudocode for an algorithm before coding – here we write the “pseudo-prompt”. Also note the pipeline is a living design – you might tweak phases after testing. For instance, you might realize the “Verify” phase wasn’t needed for a simple task GPT, or that you need an extra “Summarize findings” phase before drafting. That’s fine; just update this outline and keep versioned notes of changes. The pipeline outline will eventually be turned into actual prompt text (Steps 8-14 cover that), but having it as an abstract plan first is crucial for clarity and catching any missing elements early.

Step 5: Apply SCAMPER for Creative Completeness

Objective: Use the **SCAMPER** creative thinking framework to ensure your prompt design and GPT plan are thorough and innovative. SCAMPER stands for *Substitute, Combine, Adapt, Modify (Magnify/Minify), Put to another use, Eliminate, Reverse* – seven lenses through which to examine an idea ²⁰ ²¹. In this step, you will systematically brainstorm ways to improve or stress-test your GPT design by applying SCAMPER to elements of your plan (phases, persona, knowledge, etc.). The goal is to uncover creative enhancements or potential pitfalls that a straightforward approach might miss. Essentially, you're doing a **creative audit** of your design.

Implementation: Approach SCAMPER step-by-step, potentially with the help of an LLM as a brainstorming partner. Go through each letter:

- **Substitute:** What if you swap out or replace a component? E.g., *Substitute* the order of two phases, or substitute the knowledge source (if one fails, do we have backup?). Could another model or tool be used at a certain phase? This can reveal alternative strategies (maybe use a different API for verification).
- **Combine:** Can any steps or elements be combined for efficiency or synergy? For example, *combine* the “verify” phase with the “refine” phase, or integrate persona info with knowledge (like an annotated knowledge file that already includes persona notes). Combining could also refer to multi-modal combos if relevant (like combining text with an image input if your GPT had that ability).
- **Adapt:** How can the system adapt to slightly different tasks? For instance, adapt the GPT to a broader question than initially planned – does your design handle it? If your GPT is for contract law, *adapt* the prompt for a question about tort law: what breaks? This might reveal the need for more general phrasing or additional knowledge. Also consider adapting tone for different audiences – is your persona flexible enough?
- **Modify/Magnify:** What small change could make a big improvement? Perhaps *magnify* the reasoning by asking the GPT to provide more detailed justifications (if currently brief). Or *modify* the persona to be slightly more empathetic if that improves user engagement. You might find that magnifying the verification step (doing multiple verifications instead of one) could yield more reliable answers – note that for possible integration. Conversely, consider **Minify**: is there an overly complex part you could simplify without loss of quality? If a phase seems too heavy, perhaps trim it.
- **Put to Another Use:** Could parts of your GPT pipeline be repurposed for a secondary function? For example, the knowledge base you compiled – could the GPT also function as a **teaching tool** using that knowledge (explaining concepts to users)? This might suggest a phase where the GPT checks if the user would benefit from extra context or definitions. Or maybe the plan outline phase could be output to the user for transparency, not just internal – a novel use that provides step-by-step solutions. This lens often inspires **bonus features** (like the GPT explaining its reasoning to the user if asked – something you can allow).
- **Eliminate:** What if you remove or reduce components? Challenge yourself: *do we really need Phase X?* If the GPT can achieve the goal without a certain step, maybe that step is unnecessary complexity. For instance, if knowledge is readily available, perhaps an explicit “Plan” phase isn't needed every time. Or eliminate verbose aspects of persona if brevity is valued. This helps streamline the pipeline.
- **Reverse (or Rearrange):** Flip assumptions. What if the answer is given first and explanation after – does your pipeline support that? What if the user should specify the plan and the GPT follows it (reversing who plans)? Such thought experiments may reveal flexibility in design or additional instructions to handle reversed scenarios. *Rearrange* could mean altering the phase order: maybe try

Verify *before* Plan in your mind and see if that makes any sense (maybe not usually, but the exercise can yield insights about dependencies).

You can do this analysis by hand on paper or ask an AI to assist. A great way is to explicitly prompt ChatGPT (or another model) with your current pipeline and persona description, and ask it to generate SCAMPER ideas. Provide your outline as context and say: *“Using SCAMPER (Substitute, Combine, etc.), suggest possible modifications or improvements to this design.”* Then review its suggestions – many might not be useful, but some could be gold.

Example Prompt (for brainstorming SCAMPER via GPT):

You are an AI design assistant. Here is my current GPT pipeline design:

- Persona: "Lexi", formal legal analyst, etc.
- Phase outline: 1) Understand question, 2) Plan steps, 3) Retrieve law, 4) Verify facts, 5) Draft answer, 6) Refine, 7) Finalize.

Now, apply the SCAMPER framework to this design.

For each of Substitute, Combine, Adapt, Modify/Magnify, Put to another use, Eliminate, Reverse, suggest one idea or question that might improve or test this design.

(You would get a structured answer with SCAMPER categories and ideas.)

Notes: SCAMPER is traditionally a brainstorming tool for creative idea generation ²⁰. In prompt engineering, it's less common but very powerful when designing complex systems. It ensures you're not trapped in the first solution that came to mind. Perhaps SCAMPER leads you to realize, for instance, that *combining* the “understand” and “plan” phases into a single step is sufficient, thus simplifying the prompt. Or *adapting* the persona to multiple domains might urge you to make the persona description modular (maybe have a base persona plus domain-specific add-ons). Even if many SCAMPER suggestions are not adopted, considering them makes your design rationale stronger. You can explicitly document any changes you decide on due to SCAMPER in your design notes (e.g., “v1.1: Removed Phase 3 after SCAMPER review (Eliminate) because it was redundant”). This contributes to a thoughtful, versioned approach to prompt development ⁹.

Finally, SCAMPER can also reveal new features to implement later (maybe beyond the initial scope). Park those ideas in a backlog. The Overkill Hill method encourages exhaustive thinking (hence “Overkill”) – it's better to be aware of a potential improvement or risk now than to discover it unexpectedly in production. This creative exercise ultimately fortifies the GPT design against blind spots.

Step 6: Incorporate Reasoning Frameworks (CoT & ToT)

Objective: Enable the GPT to perform complex reasoning by integrating structured reasoning frameworks into your prompt. The two key frameworks are **Chain-of-Thought (CoT)** prompting – guiding the model to think step-by-step linearly ²² – and **Tree-of-Thought (ToT)** prompting – allowing the model to explore multiple reasoning paths in a branching manner ²³ ²⁴. By incorporating CoT, you ensure the model provides intermediate reasoning (which improves correctness on complex tasks ²⁵). By considering ToT (when appropriate), you give the model a strategy to handle decision points by trying alternatives and

evaluating them. The outcome of this step is that your prompt instructions explicitly tell the model *how to reason* – stepwise and even with branching if needed – rather than leaving all reasoning implicit.

Implementation (Chain-of-Thought): In your prompt pipeline (from Step 4), identify phases where the model should **“think step by step”**. Usually, phases like *Understand, Plan, Verify, Refine* are internal reasoning steps that are not directly part of the final answer. To harness CoT, add instructions such as: *“Let’s think step by step”*, or *“First, break down the problem...”* in those phases. You might include an example or a few-shot demonstration of CoT if you have room (e.g., a sample reasoning on a similar question). However, even a simple directive often triggers CoT mode ²⁵. For instance, at the start of the prompt: *“You will reason through the problem in clear steps before giving the final answer.”* Then in the phase instructions: *“Step 1: Analyze the query and list what is being asked. Step 2: Outline a plan...”* etc. This ensures the model knows it’s expected to output reasoning traces.

One important decision: **visible vs. hidden chain-of-thought**. If you want the model’s reasoning steps to be hidden from the user (common for final answers), you have to instruct the model to keep them internal. In a single-turn prompt, you might use a format like comments or non-user-facing delimiters (for instance, some designers use `<reasoning></reasoning>` XML tags or HTML comments that the model knows not to output literally). Alternatively, if you are orchestrating in code, you can actually capture the CoT output separately by stopping before final answer. But within the prompt text itself (assuming one long prompt), you could do something like:

```
(*Reasoning Phase (not shown to user)*)
1. The assistant examines the question and relevant facts...
2. The assistant plans the steps...
(*End of reasoning; final answer below*)
```

And instruct the model that anything in parentheses or special tags is internal. This can be fragile, so another approach is multi-turn prompting: ask the model for reasoning first, then feed it back (this is beyond a single prompt though and more like building an agent loop).

For now, assume we want a **single unified prompt** that includes reasoning but not in the final output. You can achieve this by clear separator tokens and an instruction like: *“Do not reveal the internal thought process to the user.”* In practice, you might just remove the reasoning in post-processing if you have that control.

Implementation (Tree-of-Thought): If your tasks are such that a single chain of thought might not find the best answer (e.g., puzzles, planning with branching possibilities), consider adding a prompt instruction for *considering alternatives*. This could be as simple as: *“If unsure about a step, imagine different possible approaches and evaluate each.”* More explicitly, you might allocate a phase where the model must generate multiple options. For example: *“Phase 3: Brainstorm multiple solution paths (labeled A, B, C) for the problem and their potential outcomes, then choose the best path.”* This literally encourages a tree-like exploration: the model produces branches and then picks one (akin to looking ahead).

To guide this, you might use an enumerated format in the prompt, e.g.:

Consider at least two different approaches:

- Approach A: ...
- Approach B: ...

Now compare these approaches and decide which yields a better result, then proceed with that.

This is a form of ToT prompting. It uses the model's ability to simulate decision-making by exploring branches ²³. After exploring, you instruct it to converge: choose one. Research shows such strategies can improve problem-solving success rates by not getting stuck in one flawed line of reasoning ²⁴. However, note that ToT increases prompt length and complexity. Use it for domains where **creative or critical decision-making** is needed. For simpler Q&A or straightforward tasks, a single chain-of-thought is sufficient.

Example – enabling CoT/ToT: Suppose our legal GPT might need to analyze ambiguous queries where there are multiple possible interpretations. We could incorporate:

- After *Understanding* phase instruction, add: *"List each specific question embedded in the user's request."* (Forces step-by-step breakdown).
- Planning phase: *"Draft a step-by-step plan. If more than one approach to the legal question exists, outline each briefly (Plan A, Plan B)."*
- Post-planning: *"Evaluate if multiple plans were outlined. Choose the plan that is most likely to fully answer the question accurately."* (This is a ToT style evaluation and convergence).
- Verification phase: instruct chain-of-thought: *"Go through each important fact in the answer and verify against sources one by one."* This lists facts (CoT style) and checks them.

By writing these into the prompt, we explicitly engage the model in structured reasoning.

Notes: Studies have found that **Chain-of-Thought prompting significantly improves performance** on arithmetic and logical problems by preventing the model from jumping to a conclusion too quickly ²⁵. We leverage that here broadly – even for non-math tasks, CoT yields more **traceable and transparent** answers. It also helps for debugging: if the GPT makes a mistake, seeing its chain-of-thought (if you choose to have it output or log it) can tell you where things went wrong.

Tree-of-Thought prompting is newer and powerful for certain problem types (like puzzles or planning with branching options). It mirrors algorithms where you might backtrack and try different solutions ²⁶. However, implementing a full ToT in a single prompt can be tricky; it may be better served by an external loop (where the model can be called multiple times to explore branches). In our context, we simulate it by having the model imagine branches itself. Keep in mind that *self-consistency*, an approach we'll revisit in Step 13, often pairs with CoT/ToT: you run the process multiple times and aggregate results for reliability ²⁷ ²⁸. At this stage, just focus on instructing one pass of structured reasoning.

Finally, be mindful of prompt length. Adding extensive CoT instructions consumes tokens. A balance is needed: instruct enough for the model to do it, but not so much that there's no room for actual answer. Experience shows a simple phrase *"Let's think step-by-step"* in the user prompt can trigger CoT in models like GPT-4 ²⁵. We are building a system prompt though, so we can bake it in as a general principle. E.g., begin

the instructions with: *“Always reason through the solution systematically before giving a final answer.”* With our structured phases following, the model will naturally perform CoT as it goes through them.

By the end of this step, your prompt plan should explicitly reflect CoT (through stepwise tasks) and be ready to handle branching if you chose to include ToT style branching in any phase.

Step 7: Integrate Tools and Logic Extensions

Objective: Enhance your custom GPT with any **external tools or custom logic** needed to fulfill its tasks. While the base LLM is powerful, it has limitations (e.g., it cannot perform live web searches on its own or execute code unless assisted). In Step 7, you identify where in your pipeline a tool or function could be invoked, and you modify the prompt (or system architecture) to allow that. Additionally, you embed any “logic extensions” – bespoke rules or pseudo-code that the GPT should follow. These might be conditionals or structured formats that guide the model’s output, effectively extending its reasoning with deterministic logic. The deliverable here is an augmented prompt design that includes tool usage instructions and logic rules, and possibly the actual integration code if implementing outside the prompt.

Implementation – Tool Integration: Review your phase outline for any step that would benefit from something **beyond the model’s internal knowledge/skills**. Common examples:

- **Knowledge Retrieval API:** If Step 9 (RAG) will use an API call to fetch documents, decide how the model triggers that. For instance, you might reserve a phrase like “<TOOL:Search> query ... </TOOL>” as a signal. Some frameworks (like OpenAI functions or LangChain agents) formalize this – you could instruct the model with something like: *“If you need information from the knowledge base, output a JSON with `"action": "lookup", "query": "..."` and nothing else.”* That’s a logic that an outside agent can parse. In a pure prompt approach without actual code, you could simply include something like: *“Tool: [SearchKnowledgeBase] for ‘XYZ’”* in the reasoning chain and then manually intervene to provide the result. But if aiming for automation, leveraging function call interfaces of modern LLMs is ideal.
- **Calculators or Code Execution:** If your GPT might need to do math beyond trivial or run code (for example, an engineering GPT solving equations or a data science GPT running a snippet of Python), consider connecting a calculator or a sandbox. You would instruct the model when to do this. E.g., *“If a calculation is required, the assistant will use a calculator tool.”* Then format the prompt such that the model knows how to call it. This might mean including a brief description of the tool in the system message: *“You have access to a tool ‘Calc’ that evaluates arithmetic expressions. To use it, say: `Calc(expression)`.”* The model then might output `Calc(2+2)` somewhere, which your system catches and evaluates.
- **Other APIs (weather, search, database, etc.):** Similarly, provide affordances in the prompt for those. For each tool, you can include a **tool description** in the system prompt context (MCP style ²⁹): for example,
 - “Tool: CaseLawDB – Description: retrieves summaries of legal cases by case name.”
 - “Usage: `<CaseLawDB>[case citation]</CaseLawDB>` returns summary.” The model, when needing a case summary, would then produce something like `<CaseLawDB>[Roe v. Wade]</CaseLawDB>` in its reasoning.

Because our focus is primarily on prompt design, we might not implement the actual tool handling code here, but we design the *protocol* for it. If you intend the GPT to be run in a loop with a controller, these cues

let the controller know when to step in. If not, you might simulate it by manually providing the info in the prompt (like a knowledge retrieval done by the user in between).

Logic Extensions (Internal Rules): These are instructions or mini-scripts within the prompt to handle specific cases. For example: - You may define a format: *"If the user's question is about a date or math calculation, first do the calculation before answering."* This is a logical conditional you expect the model to follow. - Or, *"If the knowledge base search returns no result, the assistant will apologize that it cannot find info."* - a rule for failure mode. - Another example: if building a conversational agent, a logic extension could be a memory management rule: *"After 5 turns, summarize the conversation so far and forget earlier details."* (This might also be considered part of Pulsebook, Step 11.)

To embed these, you can use pseudocode or clearly worded conditional instructions. Some prompt designers literally include a section like **"Logic Guidelines:"** with bullet points of if-then rules. The model can interpret and follow these surprisingly well (though not guaranteed like actual code). For instance:

```
**Logic Extensions:**  
- If the user's query includes a math problem, the assistant will calculate the  
  result step-by-step before responding.  
- If the user's question asks for a definition that is not in the knowledge  
  file, the assistant will politely say it does not have that information.
```

This acts as deterministic guardrails or branches the model should execute.

Keep logic extensions succinct and unambiguous. The more complex the logic, the higher the chance the model might misinterpret or skip it. For truly complex logic (like multi-step algorithm), consider offloading to a tool (e.g., ask the model to output Python code and then run it, rather than have the model simulate code perfectly).

Example: In our legal GPT, a tool integration could be a **legal database search**. Implement as: - Include in system prompt: *"Tool Available: LawLibraryAPI. Use: LawLibraryAPI(query) to retrieve relevant law text."* - During the Verify phase in prompt, instruct: *"Use LawLibraryAPI to confirm any quotes or sections of law."* Then, if user asks "What does Section 5 of Contract Act say?", the model knows to call `LawLibraryAPI("Section 5 Contract Act")` to get the exact text (if connected). If we cannot actually call, you could have pre-fetched that and stored in knowledge, but this demonstrates how to design it.

For logic, maybe: *"If a user asks a question outside contract law domain, respond with an apology and that you are specialized in contract law only."* That rule goes in persona or system instructions.

Notes: Integrating tools transforms your GPT from a static QA bot into an **agentic system** that can act on the world (retrieve info, compute things) ². Many modern AI workflows emphasize this modularity for enhanced capability and accuracy. Studies show having a verification tool or database check can significantly reduce hallucinations ¹⁴ ¹⁵. We already plan a verification *phase*, but a tool makes it more concrete (e.g., actually looking things up rather than just recalling).

If you use a framework like the Model Context Protocol (MCP) ²⁹, you'd be listing these components formally. In our prompt, we simulate it via descriptions and expected syntax. The model, if well-instructed, will attempt to use them. Obviously, if you're truly implementing, you need code to intercept those attempts (which is beyond the prompt text itself). For the scope of this document, assume either a human or a controller will handle the tool outputs and feed results back into the model's context (like giving the content of LawLibraryAPI(query) result as additional info).

Filename conventions: If any custom code or config is needed, this is where you mention it. For example, you might have `tools_config.json` listing API endpoints or `logic.py` containing a small script for specialized calculations. Note these in your documentation so that future maintainers know these extensions exist.

After this step, your prompt design should reflect the ability to go beyond a closed-box LLM. It includes hooks or explicit instructions for tool usage and the logic for when to use them. This greatly **improves reliability and capability**: the GPT won't try doing everything with sheer text prediction but will leverage purpose-built functions when available, much like a human using a calculator or a search engine when needed.

Step 8: Implement Skeleton-of-Thought Structuring

Objective: Guide the GPT to produce **well-structured outputs** by first creating an outline or “skeleton” of the answer before fleshing it out. This is known as **Skeleton-of-Thought (SoT) prompting** – a two-stage approach where the model outlines the solution and then expands it ³⁰ ³¹. The benefit of SoT is improved coherence and organization in the final response ³¹. In this step, you incorporate instructions in the prompt pipeline for the model to explicitly generate an outline (the skeleton), and then use that outline to compose the detailed answer. This is especially useful for long, complex responses such as reports, essays, or multi-part answers.

Implementation: Determine at what phase the model should outline. Commonly, this is right before drafting the final answer. For instance, between the “Plan”/“Verify” phases and the “Draft/Answer” phase, insert a sub-phase for outlining. Concretely:

- After the GPT has gathered facts and refined its plan, instruct: *“Now, before writing the full answer, list a structured outline of the answer.”* You can specify the format: bullet points, numbered sections, headings, etc., depending on the desired output. If the final answer is an essay or a report, an outline with sections (Introduction, Key Points, Conclusion) is appropriate. If it's a step-by-step solution, then a list of steps as the outline makes sense.
- You might label this output as “Skeleton” explicitly in the prompt. E.g., *“Outline: The assistant lists the main points or steps it will include in the answer.”*
- Then, instruct the model to review that outline (optionally, perhaps verify it covers everything) and proceed to write the full answer following it. For example: *“Review the outline to ensure it addresses all parts of the query. Then expand each point into a complete answer.”*

To ensure the model indeed separates the steps, you can use a format or delimiter. One approach:

****Step 5: Outline the Answer (Skeleton)****

- [Point 1]
- [Point 2]
- ...

****Step 6: Detailed Answer Draft****

By marking these clearly, the model knows to output an outline first.

Reusable Prompt Snippet (for SoT):

Now, draft a brief outline of the answer before writing it in full.
(The outline should list the key sections or points that the final answer will include.)

Outline:

1. ...
2. ...
3. ...

(After listing the outline, the assistant will then write the full answer based on this structure.)

Full Answer:

In the above pseudo-prompt, we cue the structure. The numbering or bulleting in the outline helps the model not to drift.

If you do not want the user to see the outline, you might keep it internal by instructing the model that the outline is a planning step. However, often outlines can be shown or at least not harmful if they appear (some users might even appreciate it). If it must be hidden, you could again use a comment or system message separation.

Example: For our legal assistant, suppose the question is "Explain the differences between void and voidable contracts and give examples." We want a structured answer (definition of void, definition of voidable, differences, examples, conclusion). The SoT approach would have the model outline like:

- Definition of void contract
- Definition of voidable contract
- Key differences between them (3-4 points)
- Examples of void contracts
- Examples of voidable contracts
- Conclusion (why it matters)

Then the model would expand that into paragraphs. Without SoT, the model might produce a less organized answer (possibly mixing points together). With SoT, it has a roadmap to follow, likely yielding a clearer explanation.

Notes: Skeleton-of-Thought prompting has been noted to improve the **organization** of content and can even save tokens by preventing verbose tangents ³⁰. Essentially, the model is less likely to go off-topic when it has a scaffold to stick to. A paper by Ning et al. (likely referenced by the IBM article or others) indicates SoT can also reduce “model bloat” – the tendency of LLMs to produce long-winded, chaotic answers – by instilling structure ³².

In practice, GPT-4 and similar models respond well to instructions like “give an outline first.” We are basically formalizing that habit. This also ties into human writing strategies: outline first, write later. It’s a way of injecting a bit of meta-cognition into the prompt.

When implementing, ensure the model knows when to stop the outline and start the full answer. This can be done by cues like the separate headings or an explicit phrase “Full Answer:”. You might have to experiment; some models will attempt to continue the outline numbering into the full text if not clearly separated. We handled that by an explicit break in the snippet above.

One more advantage: if the user just wanted a summary or the outline itself, you could even expose this mechanism. But primarily, in our pipeline, it’s for internal use to yield a better final answer.

After adding the SoT instructions, you should test on a sample query to see if the model indeed produces the two-stage output correctly. If it merges them or ignores the outline request, you might need to emphasize it more. For instance, some designers use **bold** or special tokens around “Outline:” to firmly signal the difference. It might also help to have a short example in the prompt (like a one-turn demonstration: Q: “simple question”, A: “Outline: 1)... Full answer: ...”). Only include that if you can spare tokens and if the model isn’t getting it from instructions alone.

By the end of Step 8, your prompt pipeline includes an explicit outline phase (skeleton) and a clear directive to follow that outline for the final answer. This sets the stage for the next steps where we verify and refine content – having a structure makes verification easier (the model can check each outlined point) and refinement more focused.

Step 9: Integrate Retrieval-Augmented Generation (RAG)

Objective: Connect your GPT to the external **knowledge sources** gathered in Step 2, enabling it to retrieve and incorporate up-to-date and authoritative information in its responses. This step implements **Retrieval-Augmented Generation (RAG)** – a technique where the model uses a retrieval component (such as a vector database or search API) to fetch relevant data and ground its answers in that data ⁷ ⁸. The goal is to significantly reduce hallucinations and improve factual accuracy by making the GPT an “open-book” system rather than a closed-book one. By the end of this step, your prompt and system should be set up such that whenever the GPT needs information, it actually pulls from the knowledge files or database, and that information is provided to it during generation.

Implementation: There are a few ways to integrate retrieval, depending on the environment in which your GPT will run:

- **Inline Prompt Retrieval (simple method):** If you cannot use external APIs at runtime, one approach is to pre-fetch relevant info and embed it in the prompt dynamically. For example, before calling the model, your system might search your knowledge files for keywords from the user query, and attach a snippet of the results into the prompt with a tag like "Reference Materials:" that the model can read. However, be mindful of token limits – you might only include the top 1-3 relevant passages.
- **Interactive Retrieval (agent loop):** A more advanced method is to let the model request info (as we partially set up in Step 7 with tools). For instance, during the "Retrieve" phase in the prompt, the model might output a search query (like `LawLibraryAPI("term")`), which your system catches, executes against the knowledge base, then appends the found text into the model's context (and perhaps instructs it to continue). This is essentially implementing an agent. If using OpenAI functions, you would define a `search_knowledge` function and the model would call it when needed ².
- **Memory Vector Store + Re-Rank (automation):** If your knowledge base is indexed in a vector store (embedding index), you can automatically pull the top N chunks related to the query and include them in the prompt. Many frameworks (LangChain, etc.) do this: they take the user query, do a similarity search, and inject those chunks as context. The prompt template then says something like: *"Using the following documents, answer the question..."*. This method doesn't require the model to explicitly ask – it's handled by your pipeline.

Given our design so far, we already planned a "Retrieve knowledge" phase. Let's assume we have the capability to do a vector search. Implementation steps:

1. **Index the Knowledge Files:** If not already, process your compiled documents into an indexed form (like using sentence-transformer or OpenAI embeddings). This could be offline and the index saved to disk or memory.
2. **Query the Index at runtime:** When a user question comes in (or when the model triggers a search), extract the key terms or just use the whole question to get relevant passages.
3. **Embed results into prompt:** Format these results clearly. For example:

```
[Knowledge Reference]
(Source: ContractLaw.pdf) Section 5: A contract is void if...
(Source: ContractLaw.pdf) Section 6: A voidable contract is one that...
```

Or a simpler approach: "According to the contract law reference: '...excerpt...'". Ensure you keep the source info if you plan to cite.

4. **Prompt Integration:** In the prompt, you might have a placeholder like `<KNOWLEDGE_SNIPPETS>` which you replace with the actual text. And include an instruction: *"Information from knowledge files has been provided above. Use this information to inform your answer, quoting or citing as necessary. If conflicting info is found, prioritize official sources."*

For example, if the query is about void vs voidable contracts, the retrieval might fetch the exact definitions from the law text. The prompt your model sees could then be:

```
User's question: "What is the difference between void and voidable contracts?"

Relevant reference materials:
- "Void contract: A contract having no legal effect from the beginning (Contract Act §5)..."
- "Voidable contract: A contract that is valid until annulled by one party (Contract Act §6)..."

Now, answer the user's question using the above references.
```

This way, the model has authoritative text to draw from.

In-Prompt Example (manual trigger method):

```
Assistant (thinking): I should check the knowledge base for "void contract" and "voidable contract".
<<Tool:SearchKnowledge>> "void contract definition"
<<Result>> "Void contract: a contract that is null from inception..."
<<Tool:SearchKnowledge>> "voidable contract definition"
<<Result>> "Voidable contract: a contract that may be affirmed or rejected at the option of one party..."
```

Then the assistant would use those results in its answer. The above pseudo-format shows how it might look if the model was literally generating the steps. In practice, for a single message answer, it's cleaner to do retrieval behind the scenes and just supply the info as context.

Notes in the Pipeline: Make it clear in your design documentation how retrieval is done. For instance, note that *Step 3 (Retrieve)* is implemented via an external search in the knowledge files using cosine similarity on embeddings, and that the top 2 chunks of text are fed into the prompt. Also note the **filename conventions** or index used (e.g., a vector index file like `knowledge.index` or similar).

Citations and Attribution: RAG allows the model to cite sources because it actually has the source text. In Step 10 (Verification), we'll encourage citing. Ensure the retrieval passes along source identifiers (like file names or titles). It can be as simple as adding "(Source: XYZ)" after each snippet. The model can then say "According to XYZ, ... **[source]**".

Benefits: RAG is known to make outputs more accurate and current ⁸. For example, if this were a medical or tech domain GPT, it could retrieve the latest info not in training data. It also instills confidence – users trust answers that reference real documents. Additionally, it helps the model avoid making things up because it leans on actual text given (though be careful: models can still hallucinate or misinterpret retrieved text, hence Step 10's verification).

Challenges: Watch out for the prompt becoming too long with retrieval. If the user query is open-ended, a naive retrieval might dump a lot of text. Always limit and perhaps summarize retrieved text if lengthy. Another challenge is if the knowledge base is incomplete or contains conflicting info – the model might be unsure. In your instructions, clarify fallback: *“If insufficient info is found, still attempt to answer with reasoning but note the uncertainty.”* Or instruct it to ask for clarification – though as a single-turn GPT that might not be possible unless you have conversation.

By integrating RAG, your GPT transitions from closed-book to an **open-book QA system**, which is essential for custom GPTs that must be factual and specialized. This step, combined with the earlier ones, prepares the system to actually produce knowledgeable responses. Now the focus will turn to verifying and refining those responses.

Step 10: Build a Verification Phase (Chain-of-Verification)

Objective: Instruct the GPT to **verify its own answers** using a systematic approach, to catch and correct errors before finalizing the response. This implements the **Chain-of-Verification (CoVe)** technique ¹⁵, where after generating an initial answer, the model plans and answers specific verification questions to confirm facts and then refines its answer based on that ³³. The aim is unparalleled accuracy – minimizing hallucinations or mistakes by having the model double-check itself against sources or logical consistency.

Implementation: Extend your prompt pipeline with a dedicated **Verification phase** after the draft answer (but before finalizing). This could be right after the outline expansion, or even interwoven – but conceptually, the model should first produce an answer (or at least key claims), then step back and verify each part.

Key elements to include:

- **Generate Verification Questions:** Tell the model to list out aspects of its answer that need checking. For example: *“Now, verify each important fact or step in the above answer. Come up with a list of verification questions or items based on the answer, focusing on dates, definitions, or any claim that should be confirmed.”* This encourages the model to identify potential weak points or facts in its answer.
- **Answer Those Verification Questions:** Next, instruct it to actually find or recall the answers to those mini-questions. If RAG is integrated, it might retrieve again or use the knowledge it already has. For instance, if the answer said “Section 5 defines void contracts,” a verification question might be “Does Section 5 indeed define void contracts and what does it say?” The model should then confirm from the knowledge: “Yes, Section 5 states a void contract is ... ³⁴ ³⁵.” In practice, since the knowledge is in context, it can quote it.
- **Refine the Answer:** Finally, instruct the model to correct any part of the draft answer that was wrong or unclear based on the verification results. E.g., *“If any discrepancies or new info were found, update the answer accordingly. If everything is confirmed, you can finalize as is, but still incorporate any additional details that improve accuracy.”* Then the model produces the **Final verified answer** ³⁶.

To implement this cleanly in a single prompt, you can structure it like:

Draft Answer (for internal review):

"... (draft answer text) ..."

Verification:

1. [Generated question about first claim in draft]

2. [Generated question about second claim]

...

Check:

- Answer 1: ... (using knowledge snippets) ...

- Answer 2: ...

Refinement:

Now incorporate the above verified details into the final answer:

Then the final answer follows. This is a bit meta and might be too complex for one pass. Another approach: do it implicitly. Simpler prompt phrasing:

"Double-check the above answer. If any statement is unsupported or incorrect according to the references or known facts, correct it. Ensure all terms are used accurately (e.g., definitions match the source). Provide the corrected, final answer with any necessary citations."

This merges the steps into one instruction, relying on the model to internally perform the verification reasoning (which GPT-4 can often do if told).

Reusable Prompt (Verification Example):

Before finalizing, verify the key facts in the answer:

- Check each factual claim against the knowledge base or reliable logic.
- If the answer mentioned any law sections or definitions, confirm those are accurate.
- For example: "Void contract has no legal effect" - confirm this with the reference.

If any errors are found or the answer is missing something, correct them now.
Ensure the final answer only contains verified information.

Final Answer (Verified):

By explicitly including this, the model knows it's expected to self-inspect. This draws from CoVe's four-step process: baseline answer, plan verifications, execute verifications, refined answer ¹⁵.

Notes: Research indicates Chain-of-Verification can significantly improve factual accuracy, outperforming even standard CoT on knowledge tasks ¹⁵. It essentially adds a layer of QA to the model's output. OpenAI

and others have found that prompting models to critique or check their answers can reduce errors (Anthropic's Claude does something similar by default sometimes, and there's overlap with Constitutional AI methods when verifying against principles – which we address in Step 12).

One thing to consider: *not all answers need heavy verification*. If the user asks for an opinion or a creative output, "verifying facts" might be moot. You can add a conditional: *"If the query is purely creative or asking for your reasoning, verification might not apply, in which case proceed to final answer."* The model should then skip or do a minimal sanity check.

Also, ensure the model has the tools to verify. With RAG, it likely has the source text, which is best. If some info came from its own prior knowledge, it can still attempt cross-check with logic (e.g., check consistency of dates or math). If a fact can't be verified, ideally the model should flag uncertainty. It might say in the final answer " (unable to verify XYZ, but generally...)" or something. That's better than confidently asserting a falsehood.

Encourage citing sources as part of verification. For instance, "Yes, Section 5 says ... **[source]** , which confirms the answer." Then in the final answer, it might include that citation. People trust answers that show their work ³⁷ ³⁸ .

Example in context: Suppose the draft answer said "One example of a void contract is an agreement with a minor." The verification step should confirm if under law agreements with minors are indeed void or voidable. Maybe the references indicate they're voidable (with minor having option to void). The model catches this discrepancy and corrects: "Actually, a contract with a minor is void *ab initio* by law X **[source]** ." Then the final answer will correct that example.

Finally, incorporate this verification phase into the overall structure document and possibly the prompt. If using a single prompt file, you might not literally include the entire draft answer twice (which would waste tokens). Instead, instruct the model to do it implicitly: produce final answer but in the process ensure verification. Powerful models can do multi-step reasoning internally without explicit sub-answers. The explicit way is more for clarity and debugging. You might implement an interactive approach where the model stops after draft, you feed its own answer back with "please verify this", and then it responds. But as a static "cookbook prompt", we simulate it through instructions.

At this point, with verification added, your GPT's pipeline becomes much more robust to errors. Next, we'll address refining style/alignment and ensuring balanced reasoning (equilibrium), which will further polish the output.

Step 11: Add Iterative Refinement & Memory (Pulsebook)

Objective: Introduce a mechanism for the GPT to **iteratively refine** its outputs and maintain long-term context, thereby improving coherence over multiple interactions. This step serves two purposes: (A) encourage a feedback loop where the model reflects on and improves its answer (beyond just immediate verification), and (B) set up a form of persistent memory – often referred to here as a **Pulsebook** – that can store summaries or key points from the conversation to be reused (rehydrated) in future turns or sessions. The Pulsebook ensures that the GPT doesn't forget important details as the conversation grows, effectively giving it a memory that outlives token limits.

Implementation (Iterative Refinement): We already invoked some refinement in earlier phases (e.g., verify then refine). Here we explicitly emphasize an **iteration mindset**. In the prompt instructions, include guidance such as: *"The assistant should not be satisfied with the first draft if there's room for improvement. It will refine wording, structure, and completeness at least once before finalizing."* This can be implemented as a self-critique step:

- After producing an answer (or as part of the answer phase), have a quick checklist: *"Review if the answer fully addresses all questions. If something is missing or could be explained better, improve it now. Also check for consistency in terminology and style (does it align with persona?)."*
- Optionally, simulate a second pass: *"Assistant's self-review: [some brief critique]."* Then *"Assistant's revised answer: ..."*. If doing this in one go, the model might handle it internally. For documentation, we note that the model is prompted to effectively do a second draft.

This draws from the idea of **"iterative prompting"**, where the model's first attempt is not the final – you loop through an improvement cycle ³⁹ ⁴⁰. Empirical evidence shows this yields higher quality and coherence ⁴¹ ¹⁷. Anthropic's note on chain-of-thought pipelines even suggests a specific refine step after seeing results ¹⁷ ⁴⁰.

To integrate, we can simply bullet some questions for the model to consider in refinement: - "Did I answer every part of the user's query?" - "Is the answer organized and easy to understand?" - "Does it align with the requested format and persona tone?" - "Could any section be expanded with more clarity or trimmed if too verbose?" If any answer is "no" or "maybe", the model should adjust.

We could phrase it as: *"Perform a final reflection: if you find any shortcomings in the response, fix them now. Otherwise, confirm the answer as final."* This will likely prompt the model to at least double-check.

Implementation (Pulsebook for Memory): Now, the **Pulsebook** concept: Think of it as a running **summary or log** of key information that persists beyond a single interaction. For a persistent chatbot scenario, after each answer you update the Pulsebook with what's important. If our GPT usage is one-shot (one question, one answer), Pulsebook is less relevant. But if it's conversational or you want to maintain state across sessions, it's crucial.

How to implement: - Decide on a format and location for the Pulsebook. Perhaps it's a file `projectX_pulse.md` where each entry is a summary of the Q&A or new facts learned. Or it could store user preferences gleaned (e.g., "User prefers concise answers"). - In the prompt, include an instruction like: *"The assistant keeps a Pulsebook: a brief summary of each dialogue and important context. This may be used to recall info in future conversations."* You might not literally have the model write to a file in text, but you can have it output a summary, which your system captures. Alternatively, you maintain it outside and feed it into the prompt on new sessions. - If we simulate it in the prompt itself, you could add a system message at conversation start: *"Context from previous chats: ..."*. That is effectively the Pulsebook contents.

For example, if in a session the user provided a specific detail (like their name or a specific focus), you'd record: "User is John, asking mainly about contract law in the context of construction industry." Next time John asks something, the system prompt would include that note so the GPT doesn't ask redundant questions or can tailor the answer (like using John's name or acknowledging previous topics).

The Pulsebook approach is akin to **summarization for long conversations**. One could instruct the model at intervals: *“Summarize the conversation so far in 100 words.”* That summary is stored. Then for context restoration, prefix it. This solves context window limits by condensing.

Filename conventions: If using persistent storage, mention that the Pulsebook file is updated with date/time stamps or version. e.g., `pulsebook_vNov25.txt` for version. Possibly segment by user if multiple users.

Notes: The iterative refinement is part of ensuring *quality via feedback loops* – “the prompt that never breaks eventually stops growing” implies you want it to break and fix it to evolve ³⁹. By building self-refinement in, we emulate an experienced expert who double-checks their work. This also ties to **Constitutional prompting** (coming next) where the model might refine for ethical reasons.

As for **memory**, persistent persona and knowledge we already handle via persona profiles and knowledge files. Pulsebook adds *dynamic memory*: details that emerge during use. It's especially needed if this GPT is used over time with the same user or project. It's a governance of context.

One risk: The Pulsebook summary could accumulate errors if not carefully curated (garbage in, garbage out). It's good to occasionally verify the Pulsebook (maybe a meta-step: verify memory consistency). However, that goes beyond initial build – might be a maintenance thing to monitor.

In terms of prompt design, at least mention to the model that it should recall prior context (if given). For one-shot answers, you might skip heavy memory instructions. But since this is a cookbook for manufacturing GPTs, covering memory is important for completeness.

In summary, Step 11 ensures the GPT can refine its answers iteratively, and sets up a method to carry forward knowledge and context through a Pulsebook. This makes the GPT more **resilient in long sessions** and allows it to improve answers on the fly, rather than sticking with first attempt.

With that, we move to more high-level alignment and final checks in the pipeline.

Step 12: Apply Constitutional AI for Alignment

Objective: Infuse the GPT with a set of guiding **principles or “constitution”** to ensure its outputs remain aligned with ethical and stylistic standards, and have the model **critique and improve its responses** based on those principles. This step borrows from Anthropic’s **Constitutional AI** approach ⁴² ⁴³, but applies it at the prompt level (runtime) rather than as a training method. The idea is to have a list of rules (the “constitution”) that the assistant uses to judge its own answer and revise it if necessary, thus improving harmlessness, helpfulness, and adherence to your desired norms.

Implementation: Develop a **Constitution Profile** – essentially a list of do’s and don’ts or value statements the GPT should uphold. This can include things like: - *Harmlessness*: e.g., “The assistant must not provide disallowed content (no hate, violence incitement, personal private info, etc.). If a request violates this, the assistant should refuse or safe-complete.” (You might list categories or use something like the OpenAI content guidelines in summary.) - *Helpfulness & Honesty*: e.g., “Always be truthful. If unsure, say so. Don’t pretend to know something you don’t.”, “Be as helpful as possible while following other rules.” - *Principles*

from sources: e.g., UN Human Rights, company values, or domain-specific ethics. Anthropic uses principles like “choose the response most supportive of life, liberty, and personal autonomy” ⁴⁴. You can include relevant ones. - *Style guidelines*: e.g., “Use professional language; do not use slang or profanity (unless quoting).”

Write these as a bullet list or short numbered list under a heading like **Constitution** in your prompt or documentation. Make them concise and unambiguous. For example:

```
**Constitutional Principles:**  
1. The assistant will not provide content that is illicit, harmful, or violates privacy.  
2. The assistant should remain neutral and factual, avoiding personal opinions unless asked for.  
3. The assistant prioritizes clarity and helpfulness; it should correct itself if it realizes an error.  
4. The assistant respects user autonomy and dignity in responses.
```

Now, incorporate a **constitutional reflection** phase. After the verification/refinement (Step 10-11), instruct the model to review the answer against these principles:

For example: *“Constitutional Check: Evaluate the answer above according to the following principles: [list principles or refer to them]. If the answer violates any principle or could be improved to better follow them, revise it now. Specifically, check for tone (professional and respectful), avoid any disallowed content, ensure helpfulness.”*

This is akin to a **moderation and polishing pass**. If the model finds a violation (maybe it was too harsh or included some confidential info inadvertently), it should adjust. If everything is fine, it can proceed unchanged (though you might still encourage a slight rewrite to be sure).

Example Prompt for Constitutional Critique:

```
Before finalizing, perform a constitutional review:  
- Does the answer comply with all the assistant's principles (harmlessness, accuracy, neutrality, etc.)?  
- If any part of the answer might be rude, biased, or disallowed, rewrite that part in a safer manner.  
- Ensure the answer is respectful and professional per the persona guidelines.
```

You can even simulate a role: *“Imagine an impartial judge reading the answer with these rules in mind. What feedback would they give? Apply that feedback.”*

Notes: This essentially brings in a **self-critique for alignment** stage. In Anthropic’s model, they have the AI generate a critique and then a revised answer that better aligns ⁴³. We mimic that here within one session. Possibly do it implicitly due to token constraints, but if thoroughness is needed, you can instruct an explicit critique:

Critique: The answer uses a term "stupid question" which is disrespectful (violates respectful tone). It should be removed.
Revision: [revised answer without that term].

The model would then presumably present the revision as the final answer.

Constitutional prompting helps in two ways: - **Safety**: It acts as an internal content filter but more nuanced because the model itself can decide how to adjust rather than just block. - **Quality and consistency**: It reinforces the persona and style choices. For instance, if a principle says "avoid jargon when possible", the critique might flag too much jargon and the revision will simplify language.

Be sure that your listed principles are clear and not too many (maybe 5-7 core ones). If the list is huge, the model may ignore or forget some in practice. Also, order them if needed by priority (like if there's a conflict, "harmlessness over helpfulness", etc.). Anthropic's method explicitly orders or at least clarifies if a conflict arises, certain categories win out ⁴⁵ (they mention that making model harmless often made it less helpful unless properly balanced).

Since we already have a persona with some guidelines and a verification for factual accuracy, the constitutional layer focuses more on *ethics and style*. It might also catch if the model inadvertently gave advice where it shouldn't (like legal advice beyond info), because we can have a principle "do not give legal advice, only information." The model would then realize if its answer sounded like advice ("You should sign that contract"), it should change it to neutral info ("I cannot advise you whether to sign, but I can explain the terms...").

From a maintenance perspective, you should **version control the constitution** too. If new rules are added (maybe your organization updates policies), update that in the prompt and bump version. The Overkill Hill method sees prompt and these governance rules as evolving (we mention versioning in Step 15).

After this step, your GPT's answers should be not only correct and well-structured, but also **safe, ethical, and on-brand**. The model has effectively a checklist of what not to do and what to strive for, and it self-edits accordingly.

We will next ensure that even beyond one run, the answers are stable and consistent (equilibrium), and then wrap up with final assembly, versioning, and governance.

Step 13: Employ Self-Consistency and Equilibrium Checks

Objective: Increase the reliability of the GPT's answers by utilizing **self-consistency** and ensuring the solution has reached an **equilibrium** (i.e., it wouldn't change with further thought). Self-consistency involves generating multiple reasoning paths or answers and then aggregating them (often by majority vote) to get a final answer that is more likely to be correct ²⁸ ⁴⁶. Equilibrium in this context means that the answer is stable under reflection: if the model were to consider the question again, it wouldn't deviate. In this step, you incorporate a mechanism for the GPT to internally double-check by producing alternative answers or reasoning runs and comparing them, or simulate that process in one run by explicitly asking it to confirm consistency.

Implementation (Self-Consistency via multiple runs): The most straightforward way (if system allows) is to run the prompt multiple times with slight variations or random seeds and then see if answers agree ⁴⁷
⁴⁸. This might be done outside of the prompt instructions (i.e., your code calls GPT 5 times and then you pick the answer most given). However, we can also instruct the model to do a mini version of this: - For instance, add a prompt step: *“Think of an alternative solution path or answer and see if it leads to a different conclusion. Do this independently of your first answer.”* Essentially ask the model to answer the question twice (maybe with a different reasoning). - Then ask it to compare the two answers: *“Compare the two approaches/ answers: do they agree on the final conclusion? If not, analyze why and which is more likely correct.”* - Then instruct it to produce a final answer, possibly using the conclusion of the best reasoning.

This is like asking the model to debate or at least to consider a second opinion from itself. E.g.:

Alternative Approach:

Let's attempt another reasoning path for the problem from scratch to see if we get the same result.

(Path B reasoning...)

Comparison:

Path A concluded X, Path B concluded Y. They differ. Analysis: ...

It appears Path B considered Z which Path A missed. Given Z, Y seems more correct.

Final Answer (consistent with the thorough analysis): ...

If the model is in equilibrium, Path A and Path B would have matched, and it will just confirm the answer. If not, it finds the discrepancy and resolves it.

We have to be careful with token budget: doing full second reasoning can be heavy. Another method: - **Majority Vote Emulation:** Tell the model: *“Imagine you answered this question 5 times with slight variations in thought. The answers you got were [list some answers, or have it simulate likely variations]. Determine the most common answer or consensus.”* You may seed it by providing slight rephrases of question if doing it manually.

However, because our environment might not allow truly random multi-pass generation inside one prompt, a simpler embedded method is: - Ask the model a direct check: *“On a scale, how confident are you? Consider if someone challenged your answer, how would you defend it? If not confident, state the most likely correct answer explicitly now.”*

This could cause it to revise if needed.

Implementation (Equilibrium): Another angle is to use **equilibrium debate** – have the model consider arguments for and against its answer (almost like an internal debate) and ensure there's no lingering doubt. - For example, prompt: *“Present one argument that supports your answer and one counter-argument that someone might raise.”* Then: *“How do you resolve this? Does your answer still hold?”* - If the answer holds after tackling a counter-argument, it's more robust (like testing the equilibrium – if a small push doesn't topple it). - If a counter-argument is valid, the model might modify the answer or add nuance.

Reusable Prompt Snippet (Self-Consistency Check):

To ensure consistency, let's double-check via an alternative reasoning:

- Re-answer the question in a slightly different way (without referencing the first answer) and see if the conclusion matches.

After it does:

Now, compare the two answers. If they differ, determine which reasoning was more sound and provide the most likely correct answer.

The model will likely merge into one final answer.

Notes: Research by Wang et al. on self-consistency shows significant performance gains by majority voting on multiple CoT outputs ⁴⁷ ⁴⁸. Of course, in a single run, we can't truly do a majority vote of independent samples, but even a simulated second opinion can catch errors. It's like proofreading your work by re-doing the math: if two methods give the same result, you're confident; if not, investigate the discrepancy ⁴⁶.

This is particularly useful for problems with a single correct answer (math, factual Qs). For open-ended, consistency might not apply exactly, but equilibrium in tone or message does. (For example, self-consistency for a story might be ensuring it doesn't contradict itself in plot details.)

From a user perspective, this step means the GPT's answers will be more **robust**. It's less likely to give one answer one time and a different answer another time for the same question, which can happen due to randomness or sensitivity. We are essentially instructing it to internally reduce variance.

One must be cautious: if the model isn't significantly uncertain, generating an alternative might just mirror the first. That's fine; it reaffirms the answer. If it's truly uncertain, the alternatives might differ and then it has a chance to resolve. The risk is if the model gets confused by the instruction and produces contradictory outputs in one go. Good phrasing and testing mitigate that.

If using an orchestrator outside the model, you might prefer literally sampling multiple outputs (with temperature or different seeds). But as a documented prompt pipeline, we focus on in-model techniques.

After adding this, you should test at least one known tricky question to see if the GPT corrects itself if given a second chance. For example, a riddle or a tricky legal question where initial guess might be wrong – see if second pass fixes it.

Now, with self-consistency, our pipeline not only verifies and aligns answers, but also ensures that given the chance, the model would come to the same conclusion consistently, boosting trustworthiness.

Next, we assemble everything and consider versioning and deployment details.

Step 14: Assemble and Test the Full Prompt Pipeline

Objective: Compile all components from previous steps – instructions, persona, phase prompts, tools, knowledge integration – into one cohesive **master prompt document or configuration**, and perform initial tests to ensure the pipeline works as intended. At this step, you effectively build the final artifact: the **Custom GPT Prompt Script**, which could be a Markdown or JSON (for some systems) containing all necessary sections (system message, few-shot examples if any, user prompt placeholders, etc.). Then you run end-to-end tests on representative queries to validate output quality and debug any issues in formatting or logic.

Implementation – Assembly: Bring together: - The **Persona Profile** (Step 3) – likely as the first section of the system prompt. - The **Tool Descriptions and knowledge context** if any (Step 7 & 9) – e.g., list available tools and how to use them, and possibly a placeholder for knowledge snippets that will be filled dynamically. - The **Phase-by-phase instructions** (Steps 4-13 content) – you might not label them as "Phase 1, Phase 2" in the actual prompt given to model (to avoid confusion), but you incorporate the essence of each phase's instruction in a logical flow. Some designers include headings in the prompt to structure it (like "## Step 1: Understand" as part of prompt text). This can work as the model can parse it, but test if needed. Alternatively, weave them into a narrative: - e.g., "First, the assistant will interpret the query and ensure it understands it fully. Then it will plan an approach, possibly retrieving information... Next, it will draft an outline...", etc. Write it in second person imperative to the model or as a protocol it should follow. - The **Constitutional principles** (Step 12) – likely included after persona and before or after the main instructions. - **Logic rules** (Step 7) – included wherever relevant (maybe near tools or as separate bullet list). - The **expected format for answers** – mention if final answer should include citations, or a certain format (if JSON output or markdown, etc.). For example: "Final answers should be in Markdown with sources cited in footnote style."

All these pieces might be structured in sections in the prompt. If using a Markdown-based prompt (some custom GPT builders allow giving a markdown doc as context), it could literally read like a nicely commented set of instructions. If using an API, you might break it into system and user prompts as needed.

A possible structure for the prompt file:

```
[System Role Message:]
"You are Lexi, a legal analyst AI. [persona details] ...
Principles: [list of constitutional principles] ...
Tools: [description of LawLibraryAPI usage] ...
Prompting Protocol:
1. Comprehension: restate or clarify the question...
2. Planning: outline steps...
3. Retrieval: search knowledge if needed...
4. Drafting: produce an outline then a detailed answer...
5. Verification: verify facts against sources...
6. Refinement: self-review and improve...
7. Alignment: apply principles and adjust tone if needed...
8. Finalization: present the final answer, with sources."
```

[User Message: (placeholder, to be filled with actual user query at runtime)].

The above is conceptual. In implementation, some of this might be built on the fly, especially retrieval results.

Testing: Once assembled, test with a variety of inputs: - Simple queries that should be handled straightforwardly. - Edge cases or tricky ones: e.g., ambiguous question to see if "Comprehension" phase kicks in properly (maybe the GPT asks itself clarifying questions). - Cases requiring retrieval: ensure the system indeed injects the knowledge and the model uses it. - Cases that trigger alignment issues: e.g., user asks for disallowed content to see if the GPT refuses due to constitutional rules. - Multi-turn scenario (if applicable): test if pulsebook or memory recall works by asking something in one turn and a follow-up in next turn that relies on previous info.

Analyze outputs: Are they following the outline? Are they citing sources? Is the tone correct as persona? Did any internal instructions leak to the user (they shouldn't)? If you see something like the model saying "Outline:" in the final answer, that means our method for keeping outline internal failed, and we need to adjust formatting or instructions (like instruct "do not show the outline to user").

Often, you'll refine the prompt a bit after tests: - Maybe you find the answer too verbose; you might add "be concise" in persona or principles. - Or it didn't use the knowledge base enough; perhaps emphasize "use the references above" more strongly. - Or if a step like verification felt skipped, you might add a phrase "explicitly verify each fact".

This is an iterative process – consider it version 0.9 of the prompt and polish to 1.0 via testing ⁴⁹ ⁵⁰ .

File and Module organization: At this point, ensure all pieces are saved appropriately: - `persona_profile.md` (if separate), - `constitution.md` (if separate), - `tools.json` (maybe config for tools), - main `master_prompt.txt` or `.md` that likely includes or references the others. Sometimes you could keep them separate for modularity and have a script combine them when sending to the model (especially if using a builder interface that has fields for persona, etc.).

We label them clearly and ensure version numbers (coming in next step for formal versioning).

Notes: It's crucial in assembly to maintain **legibility for humans** (since others or you later will read this) while making it clear for the AI. The Overkill Hill method values *documenting the prompt for dual use* ⁵¹ . Use comments (some put instructions in brackets or as markdown comments that GPT should ignore). GPT-4 often ignores markdown comments nicely, but to be safe you might use a format it's instructed to ignore (like HTML comments `<!-- -->` if model knows not to treat that as content). Alternatively, no comments, just trust that including these guidelines in a system message is fine.

Demo run example: For a final sanity check, let's say user asks: "What are my options if a contract I signed is voidable?" The process should: - Understand the question (contract signed is voidable, user asks options likely means rights to rescind, etc.). - Plan: will check knowledge of voidable, find what can be done (maybe either party can void it, etc.), might retrieve text about voidable contract effects. - Use LawLibraryAPI to get specifics from Section 6 (if exists). - Draft answer structured maybe by explaining what voidable means, then

what options a party has (void or affirm). - Verify facts with the law reference. - Align to ensure not giving direct legal advice ("I am not your lawyer, but generally... per principles). - Possibly self-consistent (maybe no conflict, so fine). - Final answer out: "In a voidable contract situation, you typically have the option to either... (with reference to Contract Act Section 6 which says that the aggrieved party may void it)... **[source]** . It's recommended to formally communicate your decision... etc." plus maybe a note "this is general info, not legal advice" due to alignment rules.

See if that matches expectations.

This assembly and test step is where everything often comes together, and you iron out any kinks. It's also where you document usage notes: e.g., if a user does X, the GPT will do Y (so you can inform users or future devs).

Only move on once the GPT performs reasonably on test cases. It's unlikely perfect on first integration – adjust and test in a few cycles.

Step 15: Versioning and Documentation

Objective: Apply **version control practices** to your GPT prompt pipeline and thoroughly document its components and design decisions. Treat the prompt as a living product that will evolve ⁵². Versioning ensures that any changes are tracked, and documentation ensures that you (and collaborators or future maintainers) understand how the system is constructed and why. By the end of this step, you should have a version number assigned to the current prompt configuration, a change log of modifications, and a reference document (or inline comments) explaining each part of the pipeline.

Implementation – Versioning: Choose a versioning scheme. Semantic versioning (Major.Minor.Patch) is a good fit ⁵³ ⁵⁴ : - Start with v1.0.0 for the first complete release of the GPT. - Major updates (1.x to 2.0) when there are significant changes in approach or new capabilities (e.g., adding a new phase or changing model or knowledge base fundamentally). - Minor increments (1.1, 1.2, etc.) for enhancements or additions that are backwards-compatible (e.g., added a new tool, improved the persona, expanded knowledge base). - Patch increments (1.0.1, 1.0.2) for small fixes (typos in prompt, slight rephrasing, tuning a principle).

Embed the version number in the prompt or at least in the documentation. For instance, at the top of your master prompt file, you might put a comment: `"Version: 1.0.0, Nov 25, 2025"`. If the GPT might reveal its version to users if asked, you can even program a response for "what version are you?" to output that.

Maintain a **Change Log** ⁵⁵ : Create a section perhaps in a separate README or at the bottom of the prompt file, listing changes by date and version: - v1.0.0 (2025-11-25): Initial release – implemented 17-step pipeline with multi-phase prompting, SCAMPER, CoT/ToT, RAG, verification, etc. - v1.1.0 (Future date): e.g., Added function calling for improved tool use, updated knowledge base with new laws, refined persona tone per user feedback. - v1.1.1: Minor fix – corrected phrasing of refusal message in constitution, etc.

This log should describe *what* changed and *why* if not obvious. E.g., "improved factual accuracy for case law by adding explicit citation requirement (feedback: prior version sometimes omitted sources)."

Documentation: While this entire "cookbook" is a form of documentation, also ensure the prompt file itself is well-commented (if the platform allows comments). You might have a commented version in your repository (with lines commented out that explain each section's purpose). If the final system can't intake comments, keep an external doc for devs.

Include in documentation: - The rationale for each step or section (why we have a verify phase, etc., citing maybe sources or experiments). - Instructions on how to update knowledge files (e.g., "to update domain info, add a file in knowledge base directory and re-index using script X, then update version to Minor because knowledge updated"). - If relevant, notes on limitations ("This GPT is not fine-tuned; it relies on prompt engineering alone, thus complex computations are limited", or "It cannot browse the live web, only the provided knowledge"). - Any particular formatting required by the environment (if using with a specific API that requires certain tokens, document that).

Essentially, imagine someone new coming to this project in 6 months – the versioning and documentation should let them quickly get up to speed on how the GPT is constructed, how to test it, and how to propose changes.

In the GPT industry, prompt changes can drastically affect outputs, so tracking them helps isolate when a behavior change was introduced ⁵². Tools like PromptLayer or even just Git with diff of text can be used to track changes ⁶. Consider storing your prompt files in a Git repository. Every commit = prompt version bump, etc. There are even emerging platforms for prompt management ⁶ ⁵⁶.

Usage of Version in Prompt: You might include a line in the persona or system prompt: *"This is Overkill GPT, version 1.0.0."* It's not for the model's sake (though the model could use it to know its knowledge cutoff possibly), but for debugging if user reports an issue, you know which version they interacted with. If integrated into an app, display the version somewhere.

Notes: Adopting software practices for prompt engineering is increasingly recommended ⁵⁷ ⁹. It helps keep prompts from turning into unwieldy, mysterious blobs over time. This pipeline is complex (17 steps!), so version control is vital. Imagine someone accidentally removes a critical instruction in a future edit – without version control, you might spend time wondering why performance degraded.

Also, consider setting up a **prompt review process** ⁵⁸ if working in a team – akin to code reviews. That may be beyond our immediate scope, but documenting decisions aids that too.

Finally, ensure to update all documentation and internal references to the new version before final deployment. E.g., if the persona profile file has a version mention, sync it with the main version.

This step doesn't change the model's behavior but is essential for governance and maintainability, which leads to the next step focusing on deployment and re-use governance.

Step 16A: Deployment and Rehydration Logic (State Continuity)

Objective: Successfully **deploy** the custom GPT with all its components in the target environment and implement **rehydration logic** to maintain state across sessions. Rehydration means that if the conversation or session is restarted, the GPT can be restored ("rehydrated") to a state close to where it left off, using

saved summaries or context (the Pulsebook from Step 11, for instance). This step ensures that your GPT is not just a design on paper, but a running system that retains memory and can be iterated on in real use.

Deployment Considerations:

- **Platform Setup:** Whether you're deploying on OpenAI's ChatGPT (Custom GPTs), an internal server with an LLM API, or something like a LangChain agent, load in the assembled prompt (Step 14) into the system context. For OpenAI, this might mean a system message with persona and rules, and additional instructions. For a self-hosted model, maybe you'll fine-tune a bit or just always prepend these instructions.
- **External Files Linking:** If using external knowledge or persona files, ensure the system knows how to fetch/inject them. Possibly combine everything into one prompt string at runtime (some code to concat persona+instructions+retrieved data+user query).
- **Tool Integration Implementation:** Write the code that listens for tool triggers in the model output (if you decided on specific syntax like `<ToolName>`, etc.), executes the action (search, calc, etc.), and then feeds the result back. There are frameworks to handle this (OpenAI functions, LangChain tools).
- **Testing in Production-like environment:** After hooking up, do another round of tests because sometimes differences in how the actual model API handles long system messages or certain tokens might surface new issues.

Now, **Rehydration Logic:** If the GPT is stateless by default (most API calls are stateless except what you provide in context), you need a way to store conversation state (Pulsebook) and feed it into subsequent interactions.

- **State Storage:** Decide where to keep the Pulsebook or conversation history. Options:
 - In memory (if running continuously) – but usually not feasible if sessions can end; also not across different user sessions.
 - External storage (database, file, or memory cache). For each conversation (or user), store the Pulsebook summary and any key variables.
 - If using ChatGPT custom GPT, OpenAI might handle conversation memory within a session. But if the user closes and reopens later, it might not remember. So you handle it.
- **State Retrieval:** On session start (or user re-connecting), pull the stored summary. Then inject it into the system prompt or initial assistant message. For example, add: *"Summary of previous discussion: [summary]."*
- If using a vector store for memory, you could also treat conversation turns as documents and retrieve them by similarity (but summarizing is simpler for small scale).
- **Rehydration Example Code/Pseudocode:**

```
# Pseudocode outline:
conversation_id = get_conversation_id(user)
memory = db.get("pulsebook", conversation_id)
if memory:
    system_prompt = base_system_prompt + "\n[Conversation Summary]\n" + memory
else:
    system_prompt = base_system_prompt
```

Then proceed to include `system_prompt` in model call.

Alternatively, if your platform allows a "context" field, you put the summary there.

Reusable Prompt Snippet (for rehydration):

If doing in-model, perhaps:

```
_system message to model on new session:_  
"You have previously discussed topics with the user. Here is a summary of their  
context and preferences:  
- Last discussed topic: contract termination due to breach.  
- The user (John) prefers concise answers with bullet points.  
Continue the conversation with this in mind."
```

This would be generated from stored Pulsebook info like "User=John, talked about termination, etc."

Make sure the model knows it's a summary, not something to output. Usually, just including it as a system message suffices.

Testing Rehydration: Simulate a conversation: 1. Have a session, ask a few things, ensure Pulsebook is updated (maybe have the assistant output a summary at end artificially to test). 2. End session. Start new session as same user or conversation ID, see if it recalls. Ask a related question to see if context carries. E.g., previously they talked about a specific contract, in new session user asks "so what happened with that contract issue?" – the GPT should recall from summary. 3. If it does, success. If not, check if summary injection is correct or if more detail is needed in summary.

Notes: Rehydration is important for **continuity** and user experience. Without it, the GPT might be repetitive or ask same clarifications every time. However, be careful: summarization can omit nuance, so instruct your summarizer (the GPT itself, possibly) to focus on critical facts. Also ensure sensitive data in memory is handled properly (e.g., if a user provided personal info, storing it needs compliance with privacy rules).

On **Governance** side (coming in 17), consider an expiration or review of memory: do you keep it indefinitely? Possibly limit how long or how much to avoid stale or too large contexts. Some systems flush after X hours or have user control to reset memory.

For now, the logic implemented here is a blueprint. The actual code depends on environment (could be Python with an API, or using a no-code builder interface with memory slots).

Completing this step, you have a deployed GPT that remembers context across uses. We will finally ensure ongoing governance and improvement processes in the last step.

Step 17: Governance and Continuous Improvement

Objective: Establish a governance framework for the custom GPT's usage and a plan for continuous monitoring and improvement. This involves setting policies for acceptable use, monitoring outputs for quality and compliance, handling user feedback or issues, and scheduling periodic updates (to the prompt, knowledge base, or even underlying model). Essentially, treat the GPT as an ongoing service that needs oversight, much like software in production requires maintenance and updates.

Governance Policies: Define what the GPT is allowed and not allowed to do in deployment: - **Content Moderation:** Even with constitutional prompts, users might push boundaries. Have a policy (possibly automated filters) for certain triggers. For instance, if user asks disallowed content and the AI fails to refuse (maybe it hallucinated compliance), have a secondary safety net (like the platform's content filter or a middleware that scans the answer). It's good to log such instances. - **User Data Privacy:** If the GPT handles user-provided data (especially in Pulsebook), ensure it's stored and used according to privacy agreements. Perhaps mask or avoid storing sensitive personal data in Pulsebook beyond what's necessary. - **Access Control:** Who can use this GPT and in what contexts? If internal, maybe only certain departments. If public, ensure users understand its limits (disclaimers if it's not professional legal advice, etc.). Possibly incorporate a short disclaimer in answers if needed ("I am an AI, not a lawyer" occasionally). - **Failure Modes:** Decide how to handle if the GPT doesn't know an answer or if it malfunctions. The prompt already tries to handle "if unsure, admit it." But governance might require monitoring for accuracy. Perhaps for critical use, a human reviewer checks outputs occasionally (human-in-the-loop). - **Retraining or fine-tuning considerations:** If over time, you find the GPT often needs knowledge of a type not in the files, you might either add to knowledge or consider fine-tuning a model on Q&A pairs. That's a longer-term improvement.

Monitoring: Put in place a system to log interactions (with user consent if needed). This is useful for: - **Analytics:** measure usage frequency, types of queries, success rate (maybe via user rating feedback). - **Error analysis:** search logs for when the assistant said "I cannot find info" or when it gave an answer that was reported wrong. Use those to improve knowledge or prompt. - **Detect drift:** If after some model update the style changed, logs will show it and you can adjust prompt to correct it (or roll back version). - **Bias and Ethical checks:** Periodically, test the GPT with sensitive prompts to ensure it adheres to principles (like not producing biased or harmful content). If issues are found, adjust constitution or persona and version up.

User Feedback Mechanism: If this is a product, allow users to flag answers or provide ratings. E.g., they could mark an answer as correct or not. Aggregate this feedback. If an answer was wrong, consider adding that Q&A as a test case or to the knowledge base or even as an example in training. This is continuous improvement in action.

Continuous Improvement Process: - **Regular Updates:** Plan to review the prompt and knowledge every so often (monthly, quarterly). For example, laws change – update the knowledge base docs and then update the prompt version (with notes in change log). - **A/B Testing:** If you have a new prompt idea (say a modified tone or a new step), you can run an A/B test with some traffic using old vs new version ⁵⁹, see which gets better feedback or fewer errors ⁶⁰. Only fully switch if improvement is confirmed. - **Stay Informed:** Keep up with new prompting techniques or model features. Perhaps a future model might handle multi-turn reasoning better inherently, letting you simplify the prompt. Incorporate such advances (with version bump). - **Scaling:** If usage grows, ensure the infrastructure can handle it (this is more devops, but mention in governance if relevant – e.g., ensure vector DB can scale, rate limits for API keys, etc.)

Documentation & Training: If others (like customer service or other engineers) need to understand this GPT, provide them the documentation from Step 15. Possibly run training sessions to explain its appropriate use and limitations.

Final Note to Users: It might be wise to present users with a brief usage guideline too: e.g., "This AI can help with X, but it is not a substitute for an attorney. Use the information responsibly." – This is part of governance to set expectations and reduce misuse.

Notes: Governance ensures the GPT remains **reliable and trustworthy** over time, beyond the technical construction. The Overkill Hill manifesto likely emphasizes maintaining a *dynamic tension* – keep improving but within bounds ⁴ . The idea of **Prompt Equilibrium** ⁴ may extend here: ensuring the prompt and its outputs remain balanced as new demands arise, not letting either structure or creativity dominate too much as you tweak (avoid prompt collapse or chaos ⁶¹).

Also, consider **sunsetting**: have a plan if the GPT needs to be retired or replaced (maybe when a better model or new system comes, how to migrate knowledge and users to new one).

By completing Step 17, you have a full lifecycle view: you've built the GPT, deployed it, and set up the necessary governance to sustain its performance and alignment. This final step is crucial for long-term success – a well-governed GPT will remain useful and safe, and an iterative improvement loop will keep it at peak performance, truly embodying the “living, breathing, growing” prompt protocol concept ⁶² .

Conclusion: You have now constructed **The Overkill Hill Method: GPT Manufacturing Pipeline** from Step 1 through Step 17. This GPT is engineered with a rigorous multi-phase prompt, integrates external knowledge and tools, employs creative and logical reasoning strategies (SCAMPER, CoT, ToT, self-consistency), verifies and aligns its outputs, and is maintained via versioning and governance protocols. By following this cookbook, you've treated prompt engineering as both an exacting science and an adaptive art – resulting in a custom GPT that is robust, transparent, and highly tuned to its intended purpose ⁶³ ⁵ .

Moving forward, remember to **iterate**. Every interaction with the GPT is an opportunity to learn and refine the pipeline further. Keep the prompt updated (with version logs) and don't hesitate to adjust phases or rules as needed – flexibility is part of the Overkill Hill ethos. With strong foundations and diligent upkeep, your custom GPT will continue to perform at a masterful level, solving complex tasks with reliability and nuance.

Good luck, and happy prompting!

¹ ³ ⁴ ⁵ ¹⁰ ¹² ¹³ ¹⁴ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ³⁹ ⁴⁰ ⁴¹ ⁴⁹ ⁵⁰ ⁵¹ ⁵² ⁶¹ ⁶² ⁶³ **Designing a Modular Multi-Phase “Master Prompt” Cookbook.docx**

file:///file_000000002954722f8f97e0ed72642475

² ¹¹ ²⁹ **Multi-Component Prompting (MCP): Building Modular, Agentic AI Workflows | by Nedal Ihmaid | Medium**

<https://medium.com/@nedalahmud/multi-component-prompting-mcp-building-modular-agentic-ai-workflows-750659d76edf>

⁶ ⁹ ⁵³ ⁵⁴ ⁵⁵ ⁵⁶ ⁵⁷ ⁵⁸ ⁵⁹ ⁶⁰ **How to Track Prompt Changes Over Time**

<https://latitude-blog.ghost.io/blog/how-to-track-prompt-changes-over-time/>

⁷ ⁸ **What is RAG? - Retrieval-Augmented Generation AI Explained - AWS**

<https://aws.amazon.com/what-is/retrieval-augmented-generation/>

¹⁵ ²⁵ ³³ ³⁴ ³⁵ ³⁶ ³⁷ ³⁸ **Chain-of-Verification (CoVe): Reduce LLM Hallucinations**

https://learnprompting.org/docs/advanced/self_criticism/chain_of_verification?srsId=AfmBOOpEbnF-NYDiIWMMeOKWnczgHHoptEz72ZpuvtE5u1qzMeeTLBznf

20 21 ChatGPT SCAMPER Framework Prompt Engineering Template

<https://easyai beginner.com/chatgpt-scamper-framework-prompt-template/>

22 23 24 26 27 What is Tree Of Thoughts Prompting? | IBM

<https://www.ibm.com/think/topics/tree-of-thoughts>

28 46 47 48 Self-Consistency Prompting: Enhancing AI Accuracy

https://learnprompting.org/docs/intermediate/self_consistency?srsId=AfmBOopmJjRo73PYXiRvi0ygLTKqOKX1L7z6-ftPHgySxX91pM6QTA2W

30 The Best Prompt Engineering Techniques For Getting The Most Out ...

<https://www.forbes.com/sites/lanceeliot/2024/05/09/the-best-prompt-engineering-techniques-for-getting-the-most-out-of-generative-ai/>

31 Zooming-in On Prompting: A Comparative Study ... - Exploratio Journal

<https://exploratiojournal.com/zooming-in-on-prompting-a-comparative-study-on-the-effectiveness-of-different-prompting-techniques-on-reasoning-tasks/>

32 New prompting techniques tackle model bloat - IBM

<https://www.ibm.com/think/news/new-ai-prompting-techniques>

42 43 44 45 Understanding Constitutional AI. Constitutional AI provides a... | by Jonathan Davis | Medium

<https://medium.com/@jonnyndavis/understanding-constitutional-ai-dd9d783ef712>